

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 7_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Given a BST, write an efficient function to delete a given key in it.

There are three cases that can be considered:

Deleting a node with no children: remove the node from the tree. Deleting a node with two children: Deleting a node with one child

Input Format

The first line of the input consists of an integer n representing, the number of nodes.

The second line of the input consists of integers space-separated representing, input node values.

The third line of the input consists of an integer q representing, the node value to

be deleted.

Output Format

The output prints the node values before and after the deletion of the key value in the format:

"Before deletion of key value:"

"After deleting key value:"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

10 30 5 6 7

30

Output: Before deletion of key value:

5 6 7 10 30

After deleting key value:

5 6 7 10

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *left, *right;  
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int val) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = val;  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

```

// Function to insert a value into the BST
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return createNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
        root->right = insert(root->right, val);
    return root;
}

```

```

// Helper to find the smallest node in the right subtree
struct Node* minValueNode(struct Node* node) {
    struct Node* current = node;
    while (current && current->left != NULL)
        current = current->left;
    return current;
}

```

```

// Function to delete a key from the BST
struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Case 1 & 2: Node with one child or no child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
    }
}

```

```

// Case 3: Node with two children
struct Node* temp = minValueNode(root->right);
root->data = temp->data;
root->right = deleteNode(root->right, temp->data);

```

```

    }
    return root;
}

// In-order traversal to print the tree
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    int n, val, q;
    struct Node* root = NULL;

    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }
    scanf("%d", &q);

    printf("Before deletion of key value:\n");
    inorder(root);
    printf("\n");

    root = deleteNode(root, q);

    printf("After deleting key value:\n");
    inorder(root);
    printf("\n");

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Revi is studying binary search trees (BSTs) and wants to explore all the unique BSTs that can be constructed using distinct integers from 1 to n. He decides to write a program that generates all such BSTs and prints their preorder traversals.

Write a program to ensure that the preorder traversal of all possible BSTs constructed from numbers 1 to n is printed, each on a new line.

Example

Input

3

Output

1 2 3

1 3 2

2 1 3

3 1 2

3 2 1

Explanation

Input Format

The input consists of a single integer n, representing the number of nodes.

Output Format

Each line contains the preorder traversal (space-separated) of one possible BST using values from 1 to n.

Each traversal is printed on a new line.

There should be no blank lines between the outputs, and the order of output may vary depending on the internal recursive tree generation.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 1 2 3

1 3 2

2 1 3

3 1 2

3 2 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure for the BST node
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *left, *right;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* newNode(int item) {
```

```
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
```

```
    temp->data = item;
```

```
    temp->left = temp->right = NULL;
```

```
    return temp;
```

```
}
```

```
// Function to perform Preorder Traversal
```

```
void preorder(struct Node* root) {
```

```
    if (root != NULL) {
```

```
        printf("%d ", root->data);
```

```
        preorder(root->left);
```

```
        preorder(root->right);
```

```
    }
```

```
}
```

```
// Function to generate all possible trees for range [start, end]
```

```
// This returns an array of pointers to the roots of the trees
```

```

struct Node** generateTrees(int start, int end, int* count) {
    if (start > end) {
        struct Node** list = (struct Node**)malloc(sizeof(struct Node*));
        list[0] = NULL;
        *count = 1;
        return list;
    }

    struct Node** result = NULL;
    int totalCount = 0;

    for (int i = start; i <= end; i++) {
        int leftCount, rightCount;
        // Recursively generate all left and right subtrees
        struct Node** leftSubtrees = generateTrees(start, i - 1, &leftCount);
        struct Node** rightSubtrees = generateTrees(i + 1, end, &rightCount);

        // Combine each left subtree with each right subtree using i as root
        for (int j = 0; j < leftCount; j++) {
            for (int k = 0; k < rightCount; k++) {
                struct Node* root = newNode(i);
                root->left = leftSubtrees[j];
                root->right = rightSubtrees[k];

                result = (struct Node**)realloc(result, (totalCount + 1) * sizeof(struct
Node*));
                result[totalCount++] = root;
            }
        }
        *count = totalCount;
        return result;
    }

    int main() {
        int n;
        if (scanf("%d", &n) != 1) return 0;

        if (n == 0) return 0;

        int totalTrees = 0;
        struct Node** trees = generateTrees(1, n, &totalTrees);
    }

```

```

for (int i = 0; i < totalTrees; i++) {
    preorder(trees[i]);
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Jai is fascinated by binary trees and wants to create and manipulate them. He needs your help to implement basic operations like insertion and deletion in a binary search tree.

Given a sequence of characters representing the values to be inserted into a binary search tree, followed by a character V to be deleted from the tree, your task is to construct the binary search tree and delete the specified character from it.

Your program should perform the following operations:

Construct a binary search tree from the given sequence of characters. Delete the specified character V from the constructed tree. Print the elements of the tree in sorted order after deletion.

Input Format

The first line contains an integer N, denoting the number of characters in the sequence.

The second line contains N space-separated characters, representing the sequence of characters to be inserted into the binary search tree.

The third line contains a single character V (which is guaranteed to be present in the tree), representing the value to be deleted from the binary search tree.

Output Format

The output prints a single line containing the elements of the binary search tree

in sorted order after deleting the specified character V.

If the specified value is not available in the tree, print the given input values in order traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

F B G A D

B

Output: A D F G

Answer

// You are using GCC

#include <stdio.h>

#include <stdlib.h>

// Structure for the BST node

```
struct Node {  
    char data;  
    struct Node *left, *right;  
};
```

// Function to create a new node

```
struct Node* createNode(char val) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = val;  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

// Function to insert a character into the BST

```
struct Node* insert(struct Node* root, char val) {  
    if (root == NULL) return createNode(val);  
    if (val < root->data)  
        root->left = insert(root->left, val);  
    else if (val > root->data)  
        root->right = insert(root->right, val);
```

```

    return root;
}

// Helper to find the smallest character in the right subtree
struct Node* minValueNode(struct Node* node) {
    struct Node* current = node;
    while (current && current->left != NULL)
        current = current->left;
    return current;
}

```

```

// Function to delete a character from the BST
struct Node* deleteNode(struct Node* root, char key) {
    if (root == NULL) return root;
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Node with only one child or no child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
    }
}

```

```

// Node with two children: Get the inorder successor
struct Node* temp = minValueNode(root->right);
root->data = temp->data;
root->right = deleteNode(root->right, temp->data);
}
return root;
}

```

```

// In-order Traversal to print characters in sorted order
void inorder(struct Node* root) {
    if (root != NULL) {

```

```

        inorder(root->left);
        printf("%c ", root->data);
        inorder(root->right);
    }
}

int main() {
    int n;
    char val, v;
    struct Node* root = NULL;

    if (scanf("%d", &n) != 1) return 0;

    // Insert N characters
    for (int i = 0; i < n; i++) {
        scanf(" %c", &val); // Space before %c handles whitespace
        root = insert(root, val);
    }

    // Read character to delete
    scanf(" %c", &v);

    // Delete character and print in-order traversal
    root = deleteNode(root, v);
    inorder(root);
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement:

Arun is exploring operations on binary search trees (BST). He wants to write a program that takes an unsorted array of distinct integers and constructs a BST by inserting elements in the given order (not height-balanced).

After constructing the BST, he wants to delete all nodes at the deepest level of the tree and perform a post-order traversal to print the remaining nodes.

Example

Input:

7

1 7 4 9 2 11 12

Output:

Before deleting: 2 4 12 11 9 7 1

After deleting: 2 4 11 9 7 1

Explanation:

The BST is constructed by inserting elements in the given order: 1, 7, 4, 9, 2, 11, 12. This creates a regular BST (not height-balanced) where each element is inserted according to BST properties.

The tree is traversed in post-order before deleting the deepest level, showing: 2, 4, 12, 11, 9, 7, 1.

The deepest level nodes (in this case, node 12) are then deleted.

The tree is traversed in post-order after deleting the deepest level, showing: 2, 4, 11, 9, 7, 1.

Input Format

The first line contains an integer n , representing the number of nodes to be inserted in the BST.

The second line contains n space-separated integers, representing the elements to be inserted into the BST in the given order.

Output Format

The first line prints "Before deleting: " followed by the space-separated elements of the tree in post-order traversal before deleting the deepest level, ending with a space and newline.

The second line prints "After deleting: " followed by the space-separated

elements of the tree in post-order traversal after deleting the deepest level, ending with a space and newline.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

1 7 4 9 2 11 12

Output: Before deleting: 2 4 12 11 9 7 1

After deleting: 2 4 11 9 7 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *left, *right;  
};
```

```
struct Node* createNode(int val) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = val;  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

```
struct Node* insert(struct Node* root, int val) {  
    if (root == NULL) return createNode(val);  
    if (val < root->data)  
        root->left = insert(root->left, val);  
    else if (val > root->data)  
        root->right = insert(root->right, val);  
    return root;  
}
```

```
// Function to find the maximum depth of the tree
```

```
int findMaxDepth(struct Node* root) {
```

```

    if (root == NULL) return 0;
    int leftDepth = findMaxDepth(root->left);
    int rightDepth = findMaxDepth(root->right);
    return (leftDepth > rightDepth ? leftDepth : rightDepth) + 1;
}

```

// Function to delete nodes at a specific level (depth)

```

struct Node* deleteDeepest(struct Node* root, int currentLevel, int maxDepth) {
    if (root == NULL) return NULL;

```

// If this node is at the deepest level, free it and return NULL to parent

```

    if (currentLevel == maxDepth) {
        free(root);
        return NULL;
    }

```

```

    root->left = deleteDeepest(root->left, currentLevel + 1, maxDepth);
    root->right = deleteDeepest(root->right, currentLevel + 1, maxDepth);
    return root;
}

```

```

void postorder(struct Node* root) {

```

```

    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

```

```

int main() {
    int n, val;
    struct Node* root = NULL;

```

```

    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }

```

```

    printf("Before deleting: ");
    postorder(root);
    printf("\n");

```

```
int maxDepth = findMaxDepth(root);  
// Only delete if the tree is not empty  
if (maxDepth > 0) {  
    root = deleteDeepest(root, 1, maxDepth);  
}  
  
printf("After deleting: ");  
postorder(root);  
printf("\n");  
  
return 0;  
}
```

Status : Correct

Marks : 10/10